

```
In [2]: #Array Creation Functions using Numpy:
```

```
In [3]: import numpy as np
```

```
In [4]: #Create an array from a list
```

```
In [5]: a = np.array([1,2,3])  
print("Array a:",a)
```

```
Array a: [1 2 3]
```

```
In [6]: #Create an array with evenly spaced values
```

```
In [7]: b = np.arange(0,10,2)  
print("Array b:",b)
```

```
Array b: [0 2 4 6 8]
```

```
In [8]: #Create an array filled with zeros
```

```
In [9]: d = np.zeros((2,3))  
print("Array d:\n",d)
```

```
Array d:  
[[0. 0. 0.]  
 [0. 0. 0.]]
```

```
In [10]: d = np.zeros((2,3), dtype = int)      #Convert to int  
print("Array d:\n",d)
```

```
Array d:  
[[0 0 0]  
 [0 0 0]]
```

```
In [11]: #Create an array filled with ones
```

```
In [12]: e = np.ones((3,2))  
print("Array e:\n",e)
```

```
Array e:  
[[1. 1.]  
 [1. 1.]  
 [1. 1.]]
```

```
In [13]: e = np.ones((3,2), dtype = int)      #Convert to int  
         print("Array e:\n",e)
```

```
Array e:  
[[1 1]  
 [1 1]  
 [1 1]]
```

```
In [14]: #Create an identity matrix
```

```
In [15]: f = np.eye(4)  
         print("Identity Matrix f:\n",f)
```

```
Identity Matrix f:  
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

```
In [16]: f = np.eye(4, dtype=int)           #Convert to int  
         print("Identity Matrix f:\n",f)
```

```
Identity Matrix f:  
[[1 0 0 0]  
 [0 1 0 0]  
 [0 0 1 0]  
 [0 0 0 1]]
```

```
In [17]: #Array Manipulation Functions:
```

```
In [18]: #Reshape an array:
```

```
In [19]: a1 = np.array([1,2,3])  
         reshaped = np.reshape(a1,(1,3))  
         print("Reshaped array a1:\n",a1)
```

Reshaped array a1:
[1 2 3]

```
In [20]: a2 = np.array([1,2,3,4,5,6,7,8])  
a2
```

```
Out[20]: array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
In [21]: reshaped2 = np.reshape(a2,(1,8))  
print("Reshaped array a2:\n",a2)
```

Reshaped array a2:
[1 2 3 4 5 6 7 8]

```
In [22]: #Flatten an array:
```

```
In [23]: f1 = np.array([[1,2],[3,4]])      #Convert to 1D array/Linear vector  
f1
```

```
Out[23]: array([[1, 2],  
               [3, 4]])
```

```
In [24]: flattened = np.ravel(f1)  
print("Flattened array:",flattened)
```

Flattened array: [1 2 3 4]

```
In [25]: f2 = np.array([[1,2],[3,4],[5,6],[7,8]])  
f2
```

```
Out[25]: array([[1, 2],  
               [3, 4],  
               [5, 6],  
               [7, 8]])
```

```
In [26]: flattened2 = np.ravel(f2)  
print("Flattened array:",flattened2)
```

Flattened array: [1 2 3 4 5 6 7 8]

```
In [27]: #Transpose an array:
```

```
In [28]: e1 = np.array([[1,2],[3,4]])  
e1
```

```
Out[28]: array([[1, 2],  
               [3, 4]])
```

```
In [29]: transposed = np.transpose(e1)  
print("Transposed array:\n",transposed)
```

```
Transposed array:  
[[1 3]  
 [2 4]]
```

```
In [30]: e2 = np.array([[1,2],[3,4],[5,6],[7,8]])  
e2
```

```
Out[30]: array([[1, 2],  
               [3, 4],  
               [5, 6],  
               [7, 8]])
```

```
In [31]: transposed2 = np.transpose(e2)  
print("Transposed array:\n",transposed2)
```

```
Transposed array:  
[[1 3 5 7]  
 [2 4 6 8]]
```

```
In [32]: #Stack array vertically:
```

```
In [33]: a2 = np.array([1,2])  
b2 = np.array([3,4])  
stacked = np.vstack([a2,b2])  
print("Stacked array:\n",stacked)
```

```
Stacked array:  
[[1 2]  
 [3 4]]
```

```
In [34]: #Stack array horizontally:
```

```
In [35]: a2 = np.array([1,2])
b2 = np.array([3,4])
stacked = np.hstack([a2,b2])
print("Stacked array:\n",stacked)
```

Stacked array:
[1 2 3 4]

```
In [37]: #3-Nov-2025
```

```
In [ ]: #Mathematical Functions
```

```
In [40]: #Add two arrays
g = np.array([1,2,3,4])
added = np.add(g,2)
print("Added 2 to g:",added)
```

Added 2 to g: [3 4 5 6]

```
In [41]: #Square each element
squared = np.power(g,2)
print("Squared g:",squared)
```

Squared g: [1 4 9 16]

```
In [42]: #Square root of each element
sqrt_val = np.sqrt(g)
print("Square root of g:",sqrt_val)
```

Square root of g: [1. 1.41421356 1.73205081 2.]

```
In [43]: print(a1)
print(g)
```

[1 2 3]
[1 2 3 4]

```
In [44]: #Dot product of two arrays
a2 = np.array([1,2,3])
dot_product = np.dot(a2,g)
print("Dot product of a and g:",dot_product)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[44], line 3
      1 #Dot product of two arrays
      2 a2 = np.array([1,2,3])
----> 3 dot_product = np.dot(a2,g)
      4 print("Dot product of a and g:",dot_product)

ValueError: shapes (3,) and (4,) not aligned: 3 (dim 0) != 4 (dim 0)
```

```
In [45]: print(a)
         print(a1)
```

```
[1 2 3]
[1 2 3]
```

```
In [46]: a3 = np.array([1,2,3])
         dot_product = np.dot(a1,a)
         print("Dot product of a1 and a:",dot_product)
```

Dot product of a1 and a: 14

```
In [48]: a4 = np.array([5,4,6])
         print(a4)
         print(a3)
```

```
[5 4 6]
[1 2 3]
```

```
In [49]: dot_product = np.dot(a4,a3)      #(5*1)+(4*2)+(6*3)
         dot_product
```

Out[49]: np.int64(31)

```
In [ ]: #Statistical Function:
```

```
In [50]: s = np.array([1,2,3,4])
         s
```

Out[50]: array([1, 2, 3, 4])

```
In [52]: mean = np.mean(s)
print("Mean of s:",mean)
```

Mean of s: 2.5

```
In [53]: #Standard deviation of an array
std_dev = np.std(s)
print("Standard deviation of s:",std_dev)
```

Standard deviation of s: 1.118033988749895

```
In [54]: #Minimum element of an array
minimum = np.min(s)
print("Minimum of s:",minimum)
```

Minimum of s: 1

```
In [55]: #Maximum element of an array
maximum = np.max(s)
print("Maximum of s:",maximum)
```

Maximum of s: 4

```
In [56]: #Linear Algebra Functions
#Create a matrix
matrix = np.array([[1,2],[3,4]])
matrix
```

```
Out[56]: array([[1, 2],
               [3, 4]])
```

```
In [ ]: #Random Sampling Functions
```

```
In [65]: #Generate random float values between 0 and 1
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.54488318 0.4236548 0.64589411]

```
In [58]: #Run again
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.99200209 0.03780059 0.39197996]

```
In [59]: #Run again
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.55522517 0.41525901 0.78963797]

```
In [ ]: #You can see every run the random values are changing. If I want to keep the random values same for every run then us
```

```
In [62]: #Set seed for reproducibility
np.random.seed(0)

#Generate random values between 0 and 1
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.5488135 0.71518937 0.60276338]

```
In [63]: #Run again
np.random.seed(0)

#Generate random values between 0 and 1
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.5488135 0.71518937 0.60276338]

```
In [64]: #Run again
np.random.seed(0)

#Generate random values between 0 and 1
random_vals = np.random.rand(3)
print("Random values",random_vals)
```

Random values [0.5488135 0.71518937 0.60276338]

```
In [ ]: # See everytime the values are the same.
```

```
In [67]: #Generate random integers values
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```


Random integers [7 6 8 8 1]

```
In [68]: #Run again
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```

Random integers [6 7 7 8 1]

```
In [69]: #Run again
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```

Random integers [5 9 8 9 4]

```
In [ ]: #Everytime the values are changing. Use seed() to keep the same.
```

```
In [70]: #Set seed for reproducibility
np.random.seed(0)

#Generate random integers values
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```

Random integers [5 0 3 3 7]

```
In [71]: #Run again
np.random.seed(0)

#Generate random integers values
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```

Random integers [5 0 3 3 7]

```
In [72]: #Run again
np.random.seed(0)

#Generate random integers values
random_vals = np.random.randint(0,10,5)
print("Random integers",random_vals)
```

Random integers [5 0 3 3 7]

```
In [ ]: #Boolean and Logical Functions
```

```
In [78]: #Check if all elements are true  
test1 = np.array([True,False,True])  
test1
```

```
Out[78]: array([ True, False,  True])
```

```
In [79]: all_test1 = np.all(test1)  
all_test1
```

```
Out[79]: np.False_
```

```
In [80]: test2 = np.array([False,False,False])  
test2
```

```
Out[80]: array([False, False, False])
```

```
In [81]: all_test2 = np.all(test2)  
all_test2
```

```
Out[81]: np.False_
```

```
In [85]: any_test2 = np.any(test2)  
any_test2
```

```
Out[85]: np.False_
```

```
In [82]: test3 = np.array([True,True,True])  
test3
```

```
Out[82]: array([ True,  True,  True])
```

```
In [83]: all_test3 = np.all(test3)  
all_test3
```

```
Out[83]: np.True_
```

```
In [84]: any_test3 = np.any(test3)
any_test3
```

```
Out[84]: np.True_
```

```
In [ ]: #Set Operations
```

```
In [87]: #Intersection of two arrays
set_a = np.array([1,2,3,4])
set_b = np.array([3,4,5,6])
intersection = np.intersect1d(set_a,set_b)
intersection
```

```
Out[87]: array([3, 4])
```

```
In [88]: #Union of two arrays
union = np.union1d(set_a,set_b)
union
```

```
Out[88]: array([1, 2, 3, 4, 5, 6])
```

```
In [89]: #Array Attribute Functions
#Array Attributes
a = np.array([1,2,3])

shape = a.shape
size = a.size
dimensions = a.ndim
dtype = a.dtype

print("Shape of a:",shape)
print("Size of a:",size)
print("No. of dimensions of a:",dimensions)
print("Datatype of a:",dtype)
```

```
Shape of a: (3,)
```

```
Size of a: 3
```

```
No. of dimensions of a: 1
```

```
Datatype of a: int64
```

```
In [90]: #Other Functions  
        #a = np.array([1,2,3])  
        copied_array = np.copy(a)  
        print("Copied array:",copied_array)
```

Copied array: [1 2 3]

```
In [91]: #Size in bytes of an array  
        size = a.nbytes  
        print("Size of a in bytes:",size)
```

Size of a in bytes: 24

```
In [92]: #Check if two arrays share memory  
        shared = np.shares_memory(a,copied_array)  
        shared
```

Out[92]: False

```
In [94]: copied_array1 = a[:]  
        shared1 = np.shares_memory(a,copied_array1)  
        shared1
```

Out[94]: True

```
In [ ]:
```